

Формат растрового шрифта FMK1

Версия документа: 10.07.2026

Файл `.fmk` содержит компактный монохромный растровый шрифт для MK61s mini. Текущая версия формата имеет сигнатуру FMK1, поддерживает символы Unicode BMP, моноширинные и пропорциональные глифы, а также независимый выбор сырого или RLE-представления для каждого глифа.

Этот документ описывает именно бинарный формат. Эталонные реализации находятся в `code/fmk_font.cpp`, `code/fmk_font.hpp` и `tools/fmk_font.cpp`.

Краткие ограничения

Параметр	Ограничение
Размер валидного FMK1	20..1536 байт
Кодовые точки	U+0000..U+FFFF
Диапазоны кодовых точек	1..255
Символов в одном диапазоне	1..256
Число глифов	1..65535
Максимальная ширина	1..16 пикселей
Высота	1..32 пикселя
Шаг по горизонтали	1..16 пикселей
Межстрочный интервал	0..15 пикселей

Все многобайтовые целые записываются в little-endian. Биты внутри битового потока записываются от старшего к младшему, то есть сначала маска 0x80, затем 0x40 и так далее.

Общая структура файла

```
+-----+ 0
| заголовок, 16 байт |
+-----+ 16
| диапазоны, 3 * R   |
+-----+
| битовые записи     |
| глифов без выравнивания
+-----+ size
```

R - число диапазонов из байта 10 заголовка. Первая запись глифа начинается сразу после последнего диапазона. Отдельного массива смещений нет: чтобы найти глиф с индексом N, декодер последовательно пропускает записи 0..N-1.

Заголовок

Смещение	Размер	Поле	Описание
0	4	magic	ASCII FMK1
4	1	flags	Бит 0: моноширинный шрифт; биты 1..7 равны нулю
5	1	max_width	Ширина ячейки или максимальная ширина глифа, 1..16
6	1	height	Общая высота глифов, 1..32

Смещение	Размер	Поле	Описание
7	1	advance_gap	Старшие 4 бита: default_advance - 1; младшие 4 бита: line_gap
8	2	glyph_count	Общее число глифов, little-endian
10	1	range_count	Число диапазонов, 1..255
11	1	reserved	Зарезервировано, обязательно 0
12	2	file_size	Полный размер файла, little-endian
14	2	crc16	CRC-16/CCITT-FALSE всего файла

Единственный определённый флаг:

```
FLAG_MONOSPACED = 0x01
```

Неизвестный установленный флаг делает файл недействительным. Для моношириного шрифта ширина каждого глифа равна max_width, а шаг равен default_advance; индивидуальные метрики в потоке отсутствуют. Для пропорционального шрифта max_width задаёт верхнюю границу ширины, а метрики хранятся в каждой записи.

file_size должен в точности совпадать с фактической длиной массива. Это отсекает как усечённые данные, так и дописанный хвост.

CRC-16

Используется CRC-16/CCITT-FALSE:

- начальное значение 0xFFFF;
- полином 0x1021;
- обработка без отражения битов;
- без финального XOR;
- байты 14 и 15 при расчёте считаются нулевыми.

CRC покрывает заголовок, каталог диапазонов, записи глифов и последние биты заполнения. После формирования файла конвертер записывает размер, рассчитывает CRC с нулями в поле crc16 и затем помещает результат в байты 14..15.

Псевдокод одного шага:

```
crc = 0xFFFF
for byte in file:
    if offset == 14 or offset == 15:
        byte = 0
    crc ^= byte << 8
    repeat 8 times:
        crc = (crc << 1) ^ 0x1021, if old bit 15 was 1
        crc = (crc << 1), otherwise
```

Во всех операциях crc остаётся 16-битным.

Каталог диапазонов Unicode

После заголовка идут range_count трёхбайтовых записей:

Смещение внутри записи	Размер	Поле	Описание
0	2	first	Первая кодовая точка, little-endian
2	1	count_minus_1	Число кодовых точек минус 1

Фактическое число глифов диапазона равно `count_minus_1 + 1`. Диапазоны должны быть отсортированы по `first`, не должны пересекаться и не могут выходить за `U+FFFF`. Сумма их длин должна в точности равняться `glyph_count`.

Диапазоны не содержат индексов. Индексы назначаются неявно и подряд: первый диапазон начинается с индекса 0, следующий - сразу после него. Например:

```
20 00 5E    U+0020..U+007E, 95 глифов
01 04 00    U+0401,        1 глиф
10 04 3F    U+0410..U+044F, 64 глифа
51 04 00    U+0451,        1 глиф
```

Конвертер объединяет соседние кодовые точки, но разрывает диапазон после 256 символов, поскольку длина хранится в одном байте.

Запись глифа

Записи следуют в порядке индексов и не выравниваются по границе байта. У всех глифов одна высота `height`. Пиксели идут построчно: сверху вниз, внутри строки слева направо.

Метрики пропорционального глифа

Если `FLAG_MONOSPACED` сброшен, запись начинается с 8 бит:

Биты	Значение
7..4	<code>width</code> - 1, фактическая ширина 1..16
3..0	<code>advance</code> - 1, фактический шаг 1..16

Ширина обязана быть не больше `max_width`. В моноширинном варианте этих 8 бит нет.

Сразу после метрик, либо в начале моноширинной записи, расположен бит режима `bitmap`:

- 0 - сырые пиксели;
- 1 - битовый RLE.

Сырые пиксели

После нулевого бита режима идут ровно `width * height` пикселей по одному биту. 0 означает фон, 1 - закрашенный пиксель. Строки на диске не дополняются до целого байта.

Стоимость записи `bitmap` без метрик:

```
1 + width * height бит
```

RLE

RLE-поток разворачивается ровно в `width * height` пикселей. Каждый токен начинается с бита вида.

Литеральный токен:

```
0 llll p...
```

- `llll` - 4-битное значение `length` - 1;
- длина литерала - 1..16 пикселей;
- после длины идут `length` исходных пикселей.

Токен серии:

```
1 11111 p
```

- 11111 - 5-битное значение length - 2;
- длина серии - 2..33 пикселя;
- p - единственный пиксель, повторяемый length раз.

Токены не имеют собственного признака конца. Декодер прекращает чтение, когда получил требуемое число пикселей. Токен, который переполняет bitmap, является ошибкой.

Выбор сжатия

Конвертер решает задачу минимальной стоимости динамическим программированием отдельно для каждого глифа. Для каждой позиции рассматриваются:

- литералы длиной 1..16, стоимость 5 + length бит;
- серии одинаковых битов длиной 2..33, стоимость 7 бит.

Режим auto выбирает RLE только если оптимальные RLE-токены строго короче сырых пикселей. Равная стоимость приводит к сырой записи. Общий бит режима есть в обоих вариантах и на решение не влияет.

Режимы конвертера:

Значение --compression	Поведение
auto	RLE только при реальном выигрыше, значение по умолчанию
none	Всегда сырые пиксели
rle	Всегда RLE, даже если файл станет больше

Таким образом, в одном FMK1-файле часть глифов может быть сырой, а часть - сжатой.

Завершение битового потока

После последнего пикселя последнего глифа может оставаться от 0 до 7 битов до границы байта. Все они обязаны быть нулевыми. Полный дополнительный байт заполнения запрещён: после разбора записей остаток должен быть меньше 8 бит.

Это правило вместе с file_size и CRC устраняет неоднозначный хвост и позволяет строго проверить весь файл.

Представление bitmap в RAM

Метод Face::decode выдаёт строки, выровненные до целого байта:

```
stride = (width + 7) / 8  
required = stride * height
```

Внутри строки левый пиксель соответствует 0x80. Неиспользуемые младшие биты последнего байта строки равны нулю. Максимальный буфер декодирования равен 2 * 32 = 64 байтам для глифа 16x32.

Важно различать это RAM-представление и непрерывный, не выровненный по строкам битовый поток в файле.

Строгая проверка файла прошивкой

fmk::Face::open принимает файл только после всех проверок:

- 1 Указатель не равен NULL, первичная проверка размера даёт 16..1536; после обязательного диапазона и bitmap минимальный валидный FMK1 занимает 20 байт.
- 2 Сигнатура равна FMK1, неизвестных флагов нет.

- 3 Геометрия, шаг, число глифов и число диапазонов допустимы.
- 4 Зарезервированный байт равен нулю.
- 5 Объявленный размер и CRC совпадают.
- 6 Диапазоны отсортированы, не пересекаются и покрывают ровно все глифы.
- 7 Все метрики и bitmap-потоки разбираются без выхода за конец.
- 8 Последний неполный байт содержит только нулевое заполнение.

Смещения глифов не кэшируются: поиск по кодовой точке сначала находит индекс в диапазонах, затем последовательно проходит поток до нужной записи. Это экономит RAM и подходит для небольшого лимита файла.

Конвертер FreeType

Сборка и запуск:

```
tools/build_fmfont.sh INPUT OUTPUT [options]
```

Скрипт собирает `tools/fmfont.cpp` через C++17 и `pkg-config freetype2`, если бинарник отсутствует или устарел.

Основные параметры:

Параметр	Описание
<code>--cell WxH</code>	Целевая ячейка, по умолчанию 5x8; диапазон 1x1..16x32
<code>--size N</code>	Размер FreeType в пикселях; по умолчанию $\max(8, 4 * H)$
<code>--chars SPEC</code>	Набор символов
<code>--threshold N</code>	Порог покрытия 0..255, по умолчанию 128
<code>--line-gap N</code>	Межстрочный интервал 0..15
<code>--proportional</code>	Индивидуальные ширина и шаг
<code>--compression MODE</code>	auto, none или rle

`--chars` принимает элементы через запятую:

- `ascii` - U+0020..U+007E;
- `latin1` - U+0020..U+00FF;
- `cyrillic` - Ё, А..я, ё;
- `U+XXXX` или `U+XXXX-U+YYYY`;
- произвольный UTF-8-текст.

Повторы удаляются, кодовые точки сортируются. Символы вне BMP отклоняются, а символы, отсутствующие во входном шрифте, пропускаются. Если не осталось ни одного глифа, конвертация завершается ошибкой.

Примеры:

```
tools/build_fmfont.sh font.otf font.fmk \
--cell 5x8 --chars ascii,cyrillic
```

```
tools/build_fmfont.sh font.bdf font-3x5.fmk \
--cell 3x5 --size 20 --chars U+0020-U+007E
```

```
tools/build_fmfont.sh font.ttf ui.fmk \
--cell 8x12 --proportional --compression auto
```

Растеризация

Конвертер сначала строит общий baseline-box по всем доступным запрошенным глифам. Это сохраняет взаимное вертикальное выравнивание. Затем покрытие каждого целевого пикселя вычисляется усреднением исходной маски по соответствующей прямоугольной области и сравнивается с threshold.

В моноширинном режиме сохраняется вся ячейка. В пропорциональном режиме пустые крайние столбцы обрезаются; advance устанавливается в width + 1, но не более 16 пикселей. Полностью пустой глиф получает ненулевую ширину $\max(1, \text{cell_width} / 2)$.

По умолчанию line_gap равен 1 для высоты до 5, 2 для высоты 6..8 и 0 для большей высоты.

Использование в MK61s mini

Во внутреннем хранилище FMK1 имеет тип FONT с записью F3; на виртуальном USB-диске используется расширение .fmk. Максимальный размер совпадает с лимитом формата - 1536 байт.

FMK1 уже имеет собственное сжатие. Поэтому объектное хранилище не применяет к типу FONT внешний LZSS и считает установленный флаг внешнего сжатия ошибкой. Подробности внешнего слоя приведены в MK61s-mini-Storage.md.

На графическом UC1609 файл можно просмотреть и установить до перезапуска. Отсутствующий в FMK1 символ берётся из ближайшего встроенного шрифта. Текущий рендерер использует фиксированную сетку экрана: индивидуальная ширина глифа учитывается при рисовании и центрировании, но поле advance не создаёт переменный шаг между ячейками.

На LCD1602 установка внешнего шрифта не поддерживается. Режим просмотра выбирает до восьми глифов, масштабирует их nearest-neighbor до 5x8 и временно занимает все восемь слотов CGRAM. Пропорциональный файл на LCD1602 отклоняется режимом просмотра.

Минимальный алгоритм декодирования

```
validate_header_ranges_crc()
bitpos = (16 + 3 * range_count) * 8

for each glyph:
    if proportional:
        width  = read_bits(4) + 1
        advance = read_bits(4) + 1
    else:
        width  = max_width
        advance = default_advance

    mode = read_bits(1)
    if mode == 0:
        read width * height pixels
    else:
        while produced < width * height:
            kind = read_bits(1)
            if kind == 0:
                count = read_bits(4) + 1
                read count literal pixels
            else:
                count = read_bits(5) + 2
                pixel = read_bits(1)
                emit pixel count times

require fewer than 8 remaining bits
require all remaining bits == 0
```

Декодер обязан проверять каждое сложение и длину до чтения данных. CRC не заменяет структурную проверку: корректная контрольная сумма у логически недопустимого потока не должна приводить к выходу за границы буфера.